

6 EE PROJECTS WHERE AI LIVES ON YOUR BODY

Neural interfaces. Silent speech. Intent detection. Tactile augmentation.

All built on one platform. One PCB. Six applications.

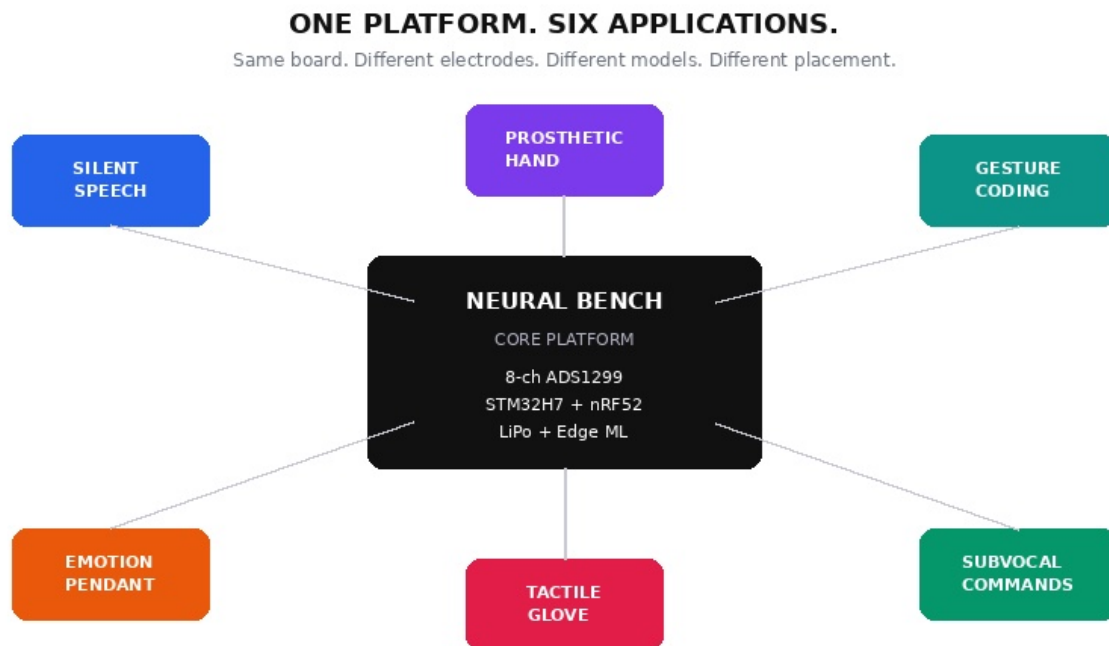
1. Silent Speech Neural Interface — talk to AI without anyone hearing you, \$60
2. EMG Prosthetic Hand — moves before your muscle contracts, \$80
3. Gesture-Controlled AR Coding — type in mid-air, anywhere, \$50
4. Emotion-Aware AI Companion — physiological context for AI, \$30
5. Tactile Augmentation Glove — feel objects before you touch them, \$45
6. Subvocal AI Commands — hands-free computing, silent input, \$35

By @1nska • Electrical Engineer

THE PLATFORM

One core. Six applications. Endless possibilities.

Every project in this guide is built on the same core hardware platform. The Neural Bench architecture combines an 8-channel 24-bit biosignal acquisition chip (ADS1299), a high-performance STM32H7 microcontroller, a Nordic nRF52 Bluetooth radio, and a LiPo charging system on a single PCB. What changes between projects is the electrode placement, the firmware configuration, and the ML model running on top. This is the philosophy: build the platform once, then build six (or sixty) applications.

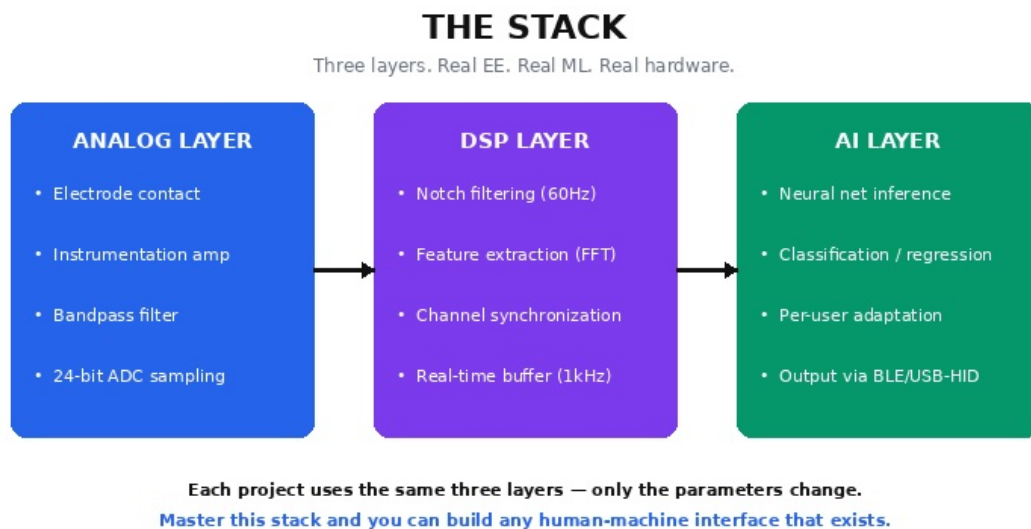


Why this matters for your portfolio: Showing a recruiter six different applications built on one platform demonstrates engineering judgment — you understand abstraction, modularity, and platform thinking. That's worth more than six unrelated demos.

THE STACK

Three layers. Master all three.

Every embodied AI project follows the same architecture. The analog layer captures biosignals from the body. The DSP layer filters and conditions those signals. The AI layer classifies or regresses against the conditioned signal to produce a decision. Understanding this stack — and being able to design every layer — is what separates a hobbyist from an engineer who can ship a neural interface product.



The analog layer is where most projects fail. Anyone can train an ML model. Few can design a clean, low-noise, multi-channel biosignal acquisition front end. The analog layer is your moat.

PROJECT 1

Silent Speech Neural Interface

Mouth words silently. AI reads your muscles. Text appears on screen.

Cost: ~\$60

Difficulty: Advanced

Build Time: 2-4 weekends

Why This Matters

Meta acquired CTRL-labs for nearly \$1 billion to build wrist-based gesture input. They have not built silent speech yet. The reason is simple: their entire platform is locked into wrist EMG, and silent speech requires reading muscle signals from the neck, jaw, and throat — a completely different body location, different muscle group, and different ML model. The acquirer for silent speech is Apple, who needs a private input method for Vision Pro and Siri, or defense contractors who need silent battlefield communication. This is the flagship project. The one that gets you noticed.

The opportunity: Silent speech has the same demo impact as 'typing without a keyboard' — but Meta isn't building it. A 3-4 year window exists before they pivot. This is the project that defines a career or a startup.

System Architecture

Silent Speech Neural Interface — Signal Chain



1000 SPS per channel | 24-bit resolution | <500ms latency mouthed-word to text
Sub-microvolt noise floor | 100dB+ CMRR | Wireless wearable form factor

Mouth words silently → muscle signals decoded → text appears on screen
Talk to AI without anyone hearing you

How It Works

- **8 EMG electrodes around the neck and jaw:** Specifically placed over the digastric, mylohyoid, geniohyoid (chin), platysma (neck), and laryngeal muscles. Each muscle activates with different phoneme groups — labial sounds (P, B, M) light up different muscles than dental sounds (T, D, N) or velar sounds (K, G).
- **ADS1299 simultaneous acquisition:** Texas Instruments' gold-standard biosignal ADC. 24-bit resolution, 8 channels sampled simultaneously at up to 16kSPS. Built-in PGA (programmable gain amp) with selectable gain from 1x to 24x. Common-mode rejection ratio >100dB. This single chip replaces what would otherwise be a discrete board with 8 instrumentation amplifiers.
- **STM32H7 DSP:** The MCU applies bandpass filtering (20-450Hz, the EMG band), notch filtering (60Hz), and computes spectral features in real-time. Output: a continuous stream of 8-channel feature vectors at 100Hz.
- **Transformer model (EMG → text):** A small encoder-decoder transformer (~2-5M parameters) trained on paired EMG-to-text data. The encoder converts the EMG feature stream into a hidden representation; the decoder generates the corresponding text. Runs on-device using TensorFlow Lite Micro or CMSIS-NN.
- **Bluetooth output:** Decoded text streams to a paired device (phone, computer, AI assistant) via BLE GATT. Latency from mouthed word to displayed text: under 500ms.

Key Design Decisions

- **Why neck/jaw electrodes instead of throat?** Throat electrodes are awkward to wear and pick up too much swallowing/breathing artifact. The neck/jaw region captures the same speech-related muscles with a wearable form factor.
- **Why 8 channels minimum?** Speech production involves dozens of muscles. 8 channels is the practical minimum to distinguish phoneme classes — fewer channels lose information; more channels add complexity without proportional accuracy gains.
- **Why a transformer instead of a CNN?** Speech is sequential. The relationship between muscle activations and meaning depends on temporal context — what came before. Transformers capture long-range dependencies that CNNs cannot.

Bill of Materials

Component	Part / Spec	Qty	~Cost
ADS1299	8-channel 24-bit biosignal ADC	1	\$25.00
STM32H7	Cortex-M7 dev board	1	\$8.00
nRF52 module	Bluetooth LE 5.0	1	\$4.00
Dry EMG electrodes	Snap-on, conductive fabric	8	\$8.00
Flex PCB	Custom, 4-layer (JLCPCB)	1	\$4.00
Neck strap mount	3D-printed TPU	1	\$2.00
LiPo battery	3.7V 500mAh	1	\$3.00
TP4056	LiPo charging IC	1	\$0.50
Misc passives	Caps, resistors, ferrites	~30	\$5.00

Build Steps

PCB design: Design a 4-layer flex PCB in KiCad. Layer stackup: signal-ground-power-signal. Route the 8 differential EMG inputs as twisted pairs with matched length. Keep digital traces (SPI between ADS1299 and STM32H7) away from analog inputs. Include a ground plane stitched with vias. Manufacture at JLCPCB — about \$4 for 5 boards.

Electrode array assembly: Sew or adhere 8 snap-on EMG electrodes onto a fabric neck collar. Reference positions: 2 under the chin (mylohyoid/geniohyoid), 2 on the lateral neck (sternocleidomastoid), 2 on the platysma, 2 near the laryngeal area. Plus 2 ground/reference electrodes. Test impedance below 5k Ω at each site.

Firmware bring-up: Configure ADS1299 via SPI: gain x12, sample rate 1kSPS, all 8 channels active. Implement a circular buffer that stores 1 second of data per channel. Apply digital filters: 20-450Hz bandpass + 60Hz notch. Verify with oscilloscope-equivalent display on connected laptop — you should see clear EMG bursts when you mouth different phonemes.

Dataset collection: Build a training dataset by recording 1000+ short utterances. Software prompts you to mouth a word silently; system records the EMG; you confirm the word. Save (EMG sequence, label) pairs. For initial proof of concept, target a vocabulary of 100 common words. Use this dataset to train your transformer.

Model training and deployment: Train an encoder-decoder transformer in PyTorch using your dataset. Architecture: 4-layer encoder, 4-layer decoder, hidden dim 128. Quantize to int8 and export to TensorFlow Lite Micro format. Deploy on STM32H7 with CMSIS-NN acceleration. Final demo: wear the device, mouth a word from your vocabulary, watch text appear on a connected phone.

Interview Questions:

- Why is the ADS1299 specifically used for biosignal acquisition — what differentiates it from a general-purpose ADC?
- How do you handle electrode-skin impedance variation across users?
- Why use a transformer model instead of CNN or RNN for EMG-to-text?
- How would you handle out-of-vocabulary words?
- What's the latency budget from mouthed word to displayed text?
- How would you scale this to a continuous dictation use case?

PROJECT 2

EMG Prosthetic Hand with Intent Detection

5-finger control from forearm muscle signals. Faster than your own reflex.

Cost: ~\$80

Difficulty: Advanced

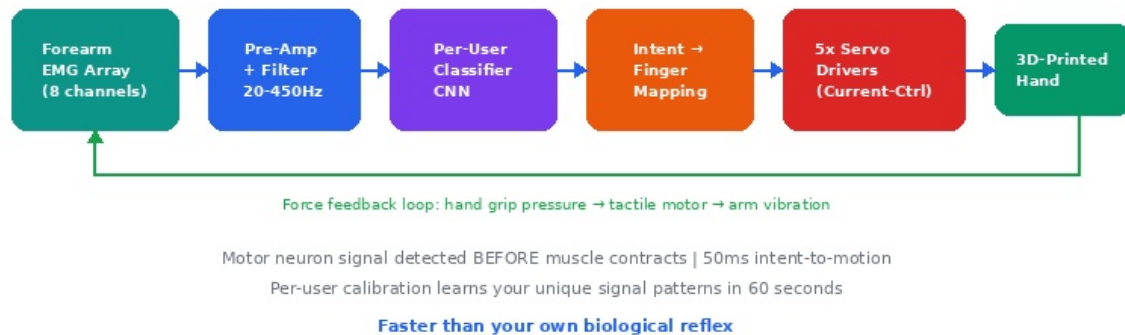
Build Time: 2-4 weekends

Why This Matters

Commercial myoelectric prosthetic hands cost \$20,000-\$50,000 and offer 2-4 grip patterns. Modern AI-driven prosthetics like the Ottobock bebionic or Open Bionics Hero Arm offer individual finger control, but at higher prices and with significant calibration requirements. This project builds a 5-finger prosthetic hand controlled by 8-channel forearm EMG with per-user CNN classification — the same architecture used in research-grade prosthetics. Accessibility is the most meaningful application of biomedical EE, and Medtronic, Abbott, and Össur all have active acquisition appetites in this space.

System Architecture

EMG Prosthetic Hand with Intent Detection — Control Loop



How It Works

- **Forearm EMG array:** Same 8-channel acquisition as the silent speech project, but with electrodes placed in a ring around the upper forearm over the flexor digitorum superficialis and extensor digitorum communis muscles. These muscles control finger flexion and extension; their electrical signals appear at the skin surface as the user attempts to move individual fingers.
- **Pre-amp + filter:** Same DSP pipeline as silent speech — 20-450Hz bandpass with 60Hz notch. Critical to remove motion artifact (slow drift below 20Hz) and electromagnetic interference (power

line noise at 60Hz).

- **Per-user classifier (CNN):** A small convolutional neural network trained on the user's specific muscle activation patterns. Every person's forearm anatomy differs, so models must be personalized. The training takes 60 seconds: the system shows the user a finger to flex; user attempts the motion; system records the EMG pattern. Repeat for all 10 motions (5 fingers × flex/extend).
- **Intent → finger mapping:** Classifier output is a probability distribution over 10 possible motions. The highest-probability motion is selected; intermediate-probability motions activate proportional finger angles for smooth, continuous control.
- **Servo drivers + 3D-printed hand:** 5 servo motors (one per finger) driven by current-controlled MOSFET drivers. Position feedback from each servo enables PID control. The hand is 3D-printed using open-source designs (Inmoov, Open Bionics) with TPU tendons providing compliance.

Bill of Materials

Component	Part / Spec	Qty	~Cost
ADS1299	8-channel biosignal ADC	1	\$25.00
STM32H7	Cortex-M7 with NN acceleration	1	\$8.00
Dry EMG electrodes	Sewn into forearm sleeve	8	\$8.00
Servo motors	Metal gear, position feedback	5	\$30.00
Servo driver board	PCA9685 16-ch PWM	1	\$3.00
3D-printed hand	PLA + TPU tendons	1	\$5.00
LiPo battery	7.4V 2000mAh (2S)	1	\$8.00
Buck converter	5V for servos, 3.3V for logic	1	\$2.00
Forearm sleeve	Compression fabric mount	1	\$3.00

Build Steps

Hand assembly: Print the open-source Inmoov or Hero Arm files in PLA. Print finger joints in TPU for flexibility. Route TPU tendons from fingertips through the palm and out the wrist, terminating at servo horns. Mount 5 servos on the forearm side of the wrist joint. Test mechanical motion: each servo should produce one finger flex when actuated.

EMG sleeve fabrication: Sew 8 EMG electrodes into a compression sleeve at the forearm positions described above. Wire each electrode to the snap connector on your Neural Bench PCB. Adjust positioning until you get clean signals on all channels when the user attempts finger movements.

Calibration session: Run the calibration software: the user wears the sleeve and attempts each of 10 finger movements (5 fingers × flex/extend) for 5 seconds each. The software records EMG signals and labels them. Total calibration time: ~60 seconds. The data is used to train the user-specific CNN.

Model training: Train a small CNN (3 conv layers + 2 dense) on the calibration data. Input: 200ms windows of 8-channel EMG (200 samples × 8 channels). Output: 10-class probability distribution. Training runs on a laptop in <2 minutes; model is then deployed to the STM32H7 via USB.

Closed-loop control test: Wear the sleeve, attempt finger movements, watch the prosthetic hand respond. Measure latency: time from EMG burst to servo motion start. Target: <100ms. If latency is too high, optimize the inference loop (reduce model size, use quantization). Test individual finger control vs. full hand grips. Document accuracy per finger.

Interview Questions:

- Why do prosthetic hand classifiers need to be per-user instead of universal?
- What's the difference between intent detection and motion detection in EMG?
- How do you handle the case where the user wears the sleeve in a slightly different position than calibration?
- Why use current-controlled servo drivers instead of standard PWM control?
- How would you add tactile feedback (sensation) for the prosthetic user?
- What FDA regulatory pathway exists for medical-grade myoelectric prosthetics?

PROJECT 3

Gesture-Controlled AR Coding Interface

Sixteen-channel wrist EMG + screen-reading camera. Type anywhere.

Cost: ~\$50

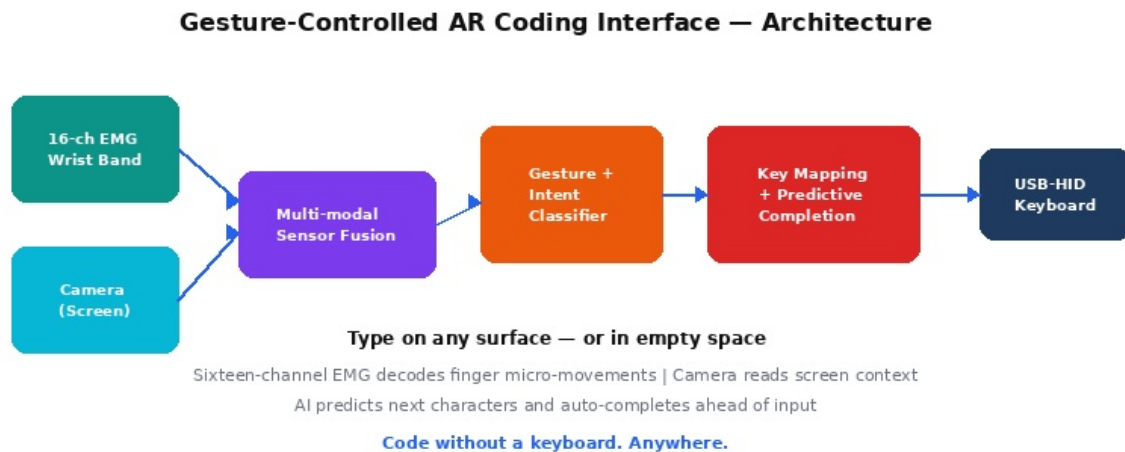
Difficulty: Advanced

Build Time: 2-4 weekends

Why This Matters

Meta's Neural Band shipped in 2025 with a vocabulary of approximately a dozen gestures. It works, but it's a closed ecosystem locked to Ray-Ban Display glasses. The next frontier is full text input — typing without a keyboard. Apple is researching this for Vision Pro. The competitive advantage: more channels (16 vs. Meta's lower count), a camera that reads screen context to enable predictive completion, and an open API that works across all devices. This is the project that demonstrates competence with both biosignal acquisition AND multi-modal AI.

System Architecture



How It Works

- **16-channel EMG wrist band:** Two stacked ADS1299 chips provide 16 simultaneous channels. The wrist band is a ring of electrodes that wraps once around the forearm just below the elbow — the location where individual finger control signals are strongest. Each finger creates a unique 16-channel pattern.
- **Camera (back of hand or fixed):** A small camera reads your screen content via OCR. This provides context for predictive completion — if you're typing in a code editor and the system sees

Python syntax, it predicts completions in Python.

- **Multi-modal sensor fusion:** Two parallel streams (EMG features + screen context) feed into a small transformer that fuses them. The EMG stream provides 'what you're trying to type'; the camera stream provides 'what you should be typing in context.'
- **Gesture + intent classifier:** Maps EMG patterns to keystrokes. Tap index finger = 'I' key. Tap middle finger = 'M' key. Pinch = 'space.' Swipe right = 'cursor right.' The mapping is learned per-user through a 5-minute calibration where the user types known sentences while wearing the band.
- **USB-HID keyboard output:** The device emulates a standard USB HID keyboard. To the host computer, it looks like any keyboard — works with any OS, any application, no special drivers required.

Bill of Materials

Component	Part / Spec	Qty	~Cost
ADS1299	16-channel total (2 chips)	2	\$50.00
STM32H7	Main processor	1	\$8.00
OV2640 camera	Small format, SPI	1	\$3.50
Dry EMG electrodes	16 ring around forearm	16	\$16.00
Flex PCB	Custom 4-layer	1	\$5.00
USB Type-C connector	For HID output	1	\$0.50
3D-printed band	TPU wrist band	1	\$2.00
LiPo battery	3.7V 500mAh	1	\$3.00

Build Steps

Dual ADS1299 setup: Cascade two ADS1299 chips via the DAISY_IN/DAISY_OUT pins. This gives 16 simultaneous channels from a single SPI bus. Configure both chips identically: gain x12, sample rate 1kSPS. Verify on a test signal that both chips sample synchronously.

Wrist band assembly: Embed 16 EMG electrodes in a TPU wrist band, spaced equally around a 60mm diameter (suitable for adult wrists). Each electrode connects to the snap header on the PCB. The 17th and 18th positions are the ground and bias drive electrodes.

Camera integration: Mount the OV2640 camera module on the back of the hand or on a desk-mounted stand pointing at the user's screen. Stream image data to STM32H7 via SPI at 10fps. Implement basic OCR (Tesseract Lite or a small CNN-based OCR) to extract text context from the camera image.

Multi-modal model training: Collect training data: user types known sentences while wearing the band. Record (EMG_features, screen_context, target_keystroke) tuples. Train a small transformer that fuses both inputs and predicts the next keystroke. Quantize and deploy to STM32H7.

HID emulation: Configure the STM32H7 USB peripheral as a HID keyboard. When the classifier outputs a keystroke prediction with high confidence, send it as a USB HID report. The host computer receives it as if from a real keyboard. Test by opening Notepad/TextEdit and 'typing' with the band on.

Interview Questions:

- Why use two cascaded ADS1299 chips instead of a single chip with more channels?
- How does the camera context improve typing accuracy?
- What's the latency from finger movement to keystroke output?
- How would you handle the case where the user types faster than the inference loop runs?
- What's the difference between USB-HID and USB-CDC for this application?
- How would you support keyboard layouts beyond QWERTY?

PROJECT 4

Emotion-Aware AI Companion Pendant

Multi-modal biosensor fusion. AI assistant adapts to your physiological state.

Cost: ~\$30

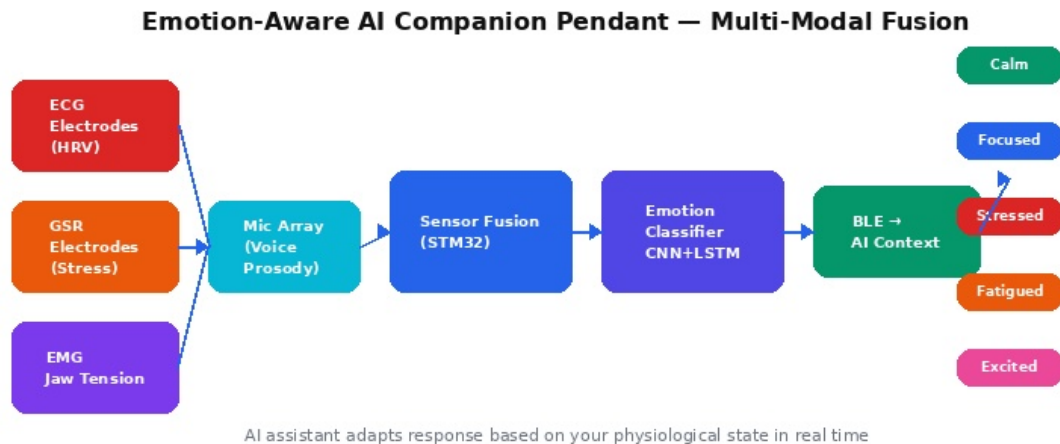
Difficulty:
Intermediate-Advanced

Build Time: 2-4 weekends

Why This Matters

AI assistants today are emotionally blind. They respond the same way whether you're calm, stressed, or exhausted. This pendant adds a physiological context channel — fusing heart rate variability, skin conductance, jaw tension, and voice prosody — to produce a continuous emotional state estimate that the AI can use. Imagine ChatGPT knowing you're stressed before you ask for help. Or your AI calendar refusing to schedule a meeting when you're fatigued. This is the missing input modality for human-AI interaction.

System Architecture



How It Works

- **Multi-modal sensor array:** The pendant houses 4 sensor channels — 2 ECG electrodes for heart rate variability (HRV), 2 GSR electrodes for skin conductance, 1 EMG channel for jaw muscle tension, and a small MEMS microphone for voice prosody analysis.
- **Sensor fusion processor:** An STM32 runs 4 parallel signal processing chains. HRV: R-peak detection from ECG, time-domain and frequency-domain HRV metrics. GSR: AC-excited conductance measurement with synchronous demod. EMG: standard biosignal pipeline. Voice: pitch tracking, formant extraction, prosody features.
- **Emotion classifier (CNN+LSTM):** A hybrid network that ingests all 4 feature streams. CNN extracts local features per stream; LSTM models temporal dynamics; final dense layer classifies into emotional states (calm, focused, stressed, fatigued, excited).
- **Bluetooth output:** The classified emotional state streams via BLE to a connected phone or AI service. APIs can subscribe to emotion state changes and adapt their responses accordingly. The OpenAI API, for example, could receive emotion as a system message context: 'User is currently stressed and fatigued.'

Bill of Materials

Component	Part / Spec	Qty	~Cost
STM32L4	Low-power MCU	1	\$5.00
AD8232	ECG analog front end	1	\$3.50
Op-amps	For GSR + EMG (TL074)	1	\$0.80
MEMS microphone	MP34DT01 (I2S)	1	\$2.00
nRF52 module	BLE 5.0	1	\$4.00
Snap electrodes	ECG + GSR + EMG	5	\$3.00
LiPo battery	3.7V 200mAh (small)	1	\$2.50
3D-printed pendant	TPU + PLA enclosure	1	\$2.00
PCB	Custom 2-layer	1	\$2.00

Build Steps

Sensor front ends: ECG: use the AD8232 dedicated ECG chip with 2 electrodes on the chest. GSR: Wheatstone bridge with 100Hz AC excitation, synchronous demod for skin conductance (see medical guide). EMG: standard pipeline for jaw muscle. Microphone: I2S MEMS mic feeds into STM32 via direct DMA.

Feature extraction: ECG → R-peak detection → HRV metrics (RMSSD, pNN50, LF/HF ratio). GSR → low-pass filter → tonic level + phasic responses. EMG → envelope detection → muscle tension level. Audio → pitch + spectral features at 100Hz frame rate.

Emotion dataset collection: Build a training dataset: ask the user to record themselves in different emotional contexts (relaxing music = calm, work stress = stressed, exercise = excited, late evening = fatigued). Capture 5-10 minutes per state. Label and store. This is a per-user dataset — emotions are personal.

Model training: Train CNN+LSTM in PyTorch using the user's labeled dataset. Architecture: 1D-CNN per stream (4 streams), then LSTM fusion, then dense classifier. Quantize and deploy. Inference runs every 5 seconds.

BLE service + AI integration: Implement a BLE GATT service that exposes the current emotional state. Build a phone app or system service that subscribes to this and passes emotion context to AI APIs. Test: have the AI assistant respond differently when you wear the pendant in different states.

Interview Questions:

- Why fuse 4 sensor modalities instead of using just one?
- What's the difference between tonic and phasic GSR responses?
- How does HRV correlate with autonomic nervous system state?
- What's the ethical consideration of streaming emotional state to AI services?
- How would you handle user calibration without manual labeling?
- What other applications exist for continuous emotion monitoring?

PROJECT 5

Tactile Sense Augmentation Glove

Feel objects from a distance. AI predicts touch before contact.

Cost: ~\$45

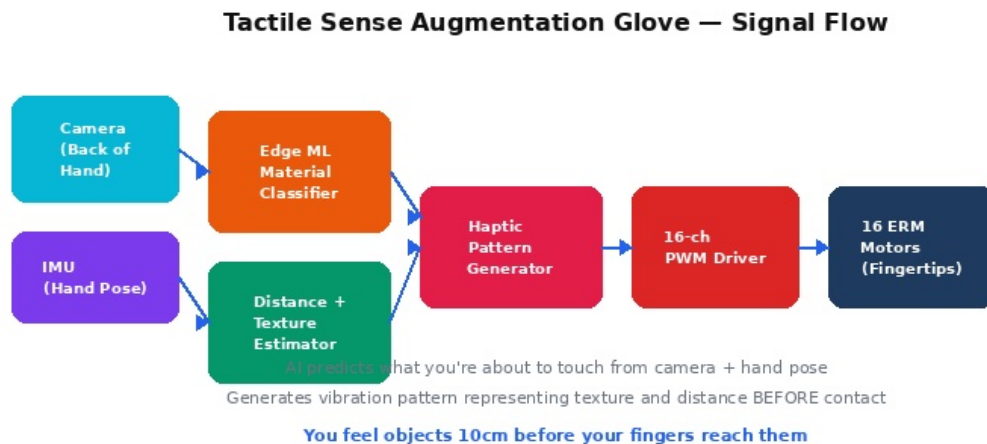
Difficulty: Advanced

Build Time: 2-4 weekends

Why This Matters

Vision and touch are biologically synchronized — you see an object and your brain predicts what it will feel like before you touch it. This glove makes that prediction explicit. A small camera identifies the material and texture of the object you're reaching for; an AI model maps that material to a haptic vibration pattern; the pattern plays on motors in the fingertips and palm. You 'feel' surfaces from 10cm away. Applications: enhanced accessibility for the visually impaired, VR/AR interaction, training for delicate manipulation tasks (surgery, archaeology, jewelry).

System Architecture



How It Works

- **Camera on the back of the hand:** A small camera (OV5640 or similar) points outward from the back of the hand, capturing whatever the hand is reaching for. Streaming at 15fps to the on-glove processor.
- **IMU for hand pose:** A 9-DOF IMU provides hand orientation and movement direction. This contextualizes the camera image — knowing the hand is reaching forward changes what counts as 'about to be touched.'

- **Material classifier:** An edge ML model (MobileNet-Tiny or similar) trained to classify materials from images: wood, metal, fabric, plastic, glass, paper, skin. Quantized and deployed on a Raspberry Pi Zero 2W or similar edge compute platform mounted on the wrist.
- **Distance + texture estimator:** A second model estimates distance to the object (using parallax from hand motion + IMU data) and surface texture (smooth, rough, grainy) from image features.
- **Haptic pattern generator:** Maps material+texture+distance to a vibration pattern. Wood = slow steady pulse on the index fingertip. Metal = sharp, high-frequency pulse on the thumb. Smooth surfaces = single steady vibration; rough surfaces = irregular pattern. Distance modulates intensity — stronger as you approach.
- **16-channel PWM motor array:** 16 ERM coin motors distributed on the fingertips (3 per finger) and palm. Each motor driven by an independent PWM channel on a PCA9685 expander. The pattern generator updates all 16 PWM values 100x per second.

Bill of Materials

Component	Part / Spec	Qty	~Cost
Raspberry Pi Zero 2W	ML inference platform	1	\$15.00
OV5640 camera	5MP, MIPI CSI	1	\$8.00
BNO055 IMU	9-DOF with fusion	1	\$10.00
PCA9685	16-channel PWM driver	1	\$3.00
ERM coin motors	10mm vibration motors	16	\$8.00
MOSFETs	2N7000 (motor drivers)	16	\$2.50
LiPo battery	3.7V 2000mAh	1	\$4.00
Glove substrate	Lycra or similar stretch fabric	1	\$3.00

Build Steps

Glove assembly: Sew 16 ERM motors into the glove substrate: 3 on each finger (proximal, middle, distal phalanges), 1 on the palm. Route thin wires from each motor along the glove's seams back to the PCB at the wrist. Use heat-shrink to insulate connections.

Computing module mount: Mount the Pi Zero 2W on the wrist with a 3D-printed bracket. Connect the OV5640 camera via flat ribbon to the Pi's CSI port. Connect the BNO055 IMU via I2C. Connect the PCA9685 via I2C to the same bus. Power everything from the LiPo through a buck converter.

Material classification model: Train MobileNetV3-Small on a custom dataset of material images (use Open Images dataset for initial training, fine-tune on your own samples). 7 classes: wood, metal, fabric, plastic, glass, paper, skin. Quantize to int8 and deploy via TensorFlow Lite on the Pi.

Haptic pattern library: Define 7 distinct vibration patterns (one per material). Each pattern is a 1-second sequence of 16-channel PWM values. Patterns should feel distinct enough that the user learns to associate them with materials within minutes of use. Test with blindfolded subjects.

Real-time loop: Main loop runs at 15fps: capture frame → classify material → estimate distance → look up haptic pattern → modulate by distance → write PWM values to PCA9685. Total latency: <100ms from camera frame to haptic feedback.

Interview Questions:

- Why use vibration motors instead of pneumatic or piezo actuators?
- How do you handle multiple objects in the camera view?
- What determines the spatial resolution of your haptic feedback?
- How would you train the user to associate vibration patterns with materials?
- What's the difference between this and actual texture sensing?
- How does the IMU contribute to distance estimation?

PROJECT 6

Subvocal AI Command Interface

4 electrodes under the chin. 50 commands. Always-on. Hands-free.

Cost: ~\$35

Difficulty:
Intermediate-Advanced

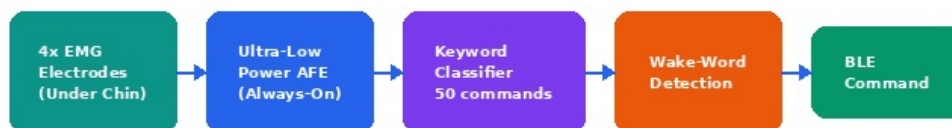
Build Time: 2-4 weekends

Why This Matters

Silent speech for full dictation is the moonshot. Subvocal commands for everyday computing is the immediate product. A small 4-electrode patch under the chin runs always-on classification of 50 common commands: 'open email,' 'send message,' 'play music,' 'navigate home,' 'what time.' Mouth the command silently — your phone or AI assistant responds within 100ms. This is the practical version that ships in 6 months instead of 3 years. Apple, Google, and Amazon would all want this for their voice assistants.

System Architecture

Subvocal AI Command Interface — Always-On Architecture



Example commands (silent sub-vocalizations):

"open email" · "send message" · "play music" · "navigate home" · "what time"

Mouth a command without sound — your phone responds within 100ms

Hands-free computing with zero audible sound

How It Works

- **4 EMG electrodes under the chin:** Strategic placement over the digastric and mylohyoid muscles — the muscles involved in jaw movement during speech. 4 channels are sufficient for a fixed vocabulary of commands (unlike full speech which requires 8+).
- **Ultra-low-power analog front end:** Optimized for always-on operation. The MCU sleeps between samples; an analog wake-up circuit (envelope detector + comparator) triggers full processing only when subvocal activity exceeds threshold. Average power draw: <1mA, enabling all-day battery life from a small coin cell.
- **Keyword classifier:** A small CNN trained on the fixed vocabulary of 50 commands. Output: 50-class probability distribution plus a 'silence' class. Inference runs only when wake-up triggers — typically 100-500 times per day.
- **Wake-word detection:** The first word of any valid command serves as the wake-word. If 'Hey' or 'OK' or your custom trigger is detected, the system listens for the rest of the command. This prevents false activations from random mouth movements.
- **BLE command output:** Detected commands transmit via BLE to your phone, smart watch, or AI assistant. Standard intents like 'open_email' or 'play_music' map directly to OS-level actions.

Bill of Materials

Component	Part / Spec	Qty	~Cost
STM32L0	Ultra-low-power MCU	1	\$3.00
ADS1294R	4-channel biosignal ADC	1	\$12.00
Dry electrodes	Adhesive, low-profile	4	\$3.00
nRF52 module	BLE 5.0	1	\$4.00
CR2032 battery	Coin cell, long-life	1	\$1.00
Flex PCB	Custom, ultra-thin	1	\$3.00
Adhesive patch	Medical-grade, skin-safe	1	\$2.00
Charging coil	Wireless charging (optional)	1	\$3.00

Build Steps

Form factor design: Design a flexible PCB shaped like a small bandage (2cm x 4cm). 4 electrode pads on the underside. Battery and electronics on the upper side. The whole assembly is meant to be worn on the underside of the chin, held by adhesive or a thin neck strap. Manufacture with JLCPCB's flex PCB service.

Wake-up circuit: Build an analog wake-up: envelope detector on one EMG channel feeds a comparator with hysteresis. When signal exceeds threshold, comparator output wakes the MCU from sleep. This eliminates the need to run the classifier continuously — saving 99% of the power budget.

Command dataset: Define 50 commands covering common phone/AI assistant interactions. Record 20+ examples per command from the user. Use data augmentation (time shifts, gain variations) to expand the dataset 10x. Total training set: 10,000+ examples.

Classifier training: Train a small CNN: 3 conv layers + 1 dense + 1 softmax. Input: 1-second 4-channel windows. Output: 51 classes (50 commands + silence). Target accuracy: >95%. Quantize to int8 for deployment. Model size: <100KB.

Integration with phone OS: Build an Android service or iOS app that listens for BLE commands from the device. Each command maps to an OS-level intent: 'open_email' launches the email app, 'send_message' opens a compose window with the last contact, etc. Test the full pipeline: wear the device, mouth a command silently, watch your phone respond.

Interview Questions:

- Why use 4 channels instead of 8 for this application?
- How does the analog wake-up circuit save power?
- What's the false activation rate and how do you minimize it?
- How would you add new commands without retraining the whole model?
- What's the latency budget from mouthed command to phone action?
- How would you handle multi-language support?

The body is the next interface.

The circuit is the bridge.

Six projects. One platform. The same core hardware — ADS1299, STM32H7, nRF52 — builds all of them. What changes is the electrode placement, the firmware configuration, and the ML model. This is platform thinking: design once, deploy infinitely. It's how Apple designed the iPhone (one device, infinite apps), how Boston Dynamics designed Spot (one robot, infinite tasks), and how every successful hardware product scales.

Meta paid nearly a billion dollars for one of these applications. Apple is racing to build another. The window for an independent builder is short — but it's open. Build something with this guide and you're not just decorating a portfolio. You're entering a market.

Build even one of these. Document the design. Screenshot the waveforms. Open-source the demo (but keep the dataset and model architecture private). Put it on your GitHub with a proper README. You'll have a portfolio piece that recruiters and investors actually remember.

One platform. Six applications. The future of embodied AI.

Follow @1nska on Instagram for build logs, new guides, and projects at the edge of what's buildable.

— Enes (@1nska) • Electrical Engineer